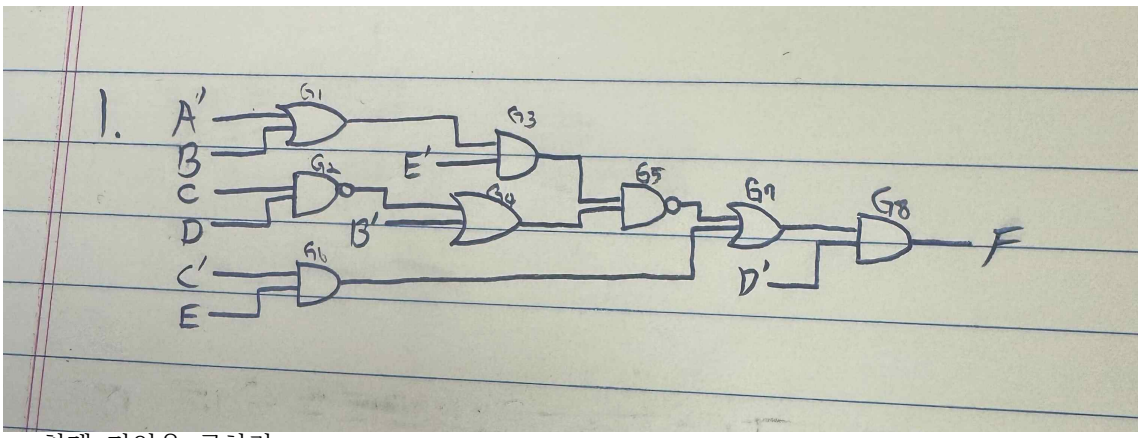
디지털 논리 회로 과제3

23013463 이우석

1. 아래의 회로를 보고 문제를 풀어라.



a. 최대 지연을 구하라.

그럼 내가 낸 문제유형과 비슷하게 어디서도 출제되지 않은 문제를 만들어서 풀이도 정확히 해줘서 알려줘.

2초 동안 생각함

좋아.
네가 낸 문제랑 **같은 유형**으로, 즉

- 회로도를 보고
- **전체 최대 지연**
- **보수/비보수 입력 모두 가능할 때**
- **비보수만 가능해서 인버터를 추가해야 할 때**
- **특정 입력부터 출력까지의 최대 지연**

을 묻는 **새 문제**를 하나 만들어서, 바로 정확하게 풀이해줄게.

새 문제

다음 조합회로가 있다. 모든 게이트의 지연은 동일하게 τ이다.

회로는 아래와 같이 연결되어 있다.

회로 구성

회로 구성

- 게이트 G_1: 입력 A', B 를 받는 OR 게이트
- 게이트 G_2: 입력 C, D 를 받는 NAND 게이트
- 게이트 G_3: 입력 G_1의 출력, E' 를 받는 AND 게이트
- 게이트 G_4: 입력 G_2의 출력, B' 를 받는 OR 게이트
- 게이트 G_5: 입력 G_3의 출력, G_4의 출력을 받는 NAND 게이트
- 게이트 G_6: 입력 C', E 를 받는 AND 게이트
- 게이트 G_7: 입력 G_5의 출력, G_6의 출력을 받는 OR 게이트
- 게이트 G_8: 입력 D', G_7의 출력을 받는 AND 게이트

최종 출력은 F이다.

문제

1. 회로의 전체 최대 지연을 구하라.
   (1) 모든 입력은 보수와 비보수 모두 사용 가능하다고 가정
   (2) 비보수 입력만 가능하고, 보수 입력은 인버터를 추가하여 만들어야 한다고 가정
2. 입력 C부터 출력 F까지의 최대 지연을 구하라.
   단, 모든 입력은 보수와 비보수 모두 사용 가능하다고 가정한다.

풀이

먼저 회로를 단계적으로 보자.

1단계: 각 게이트까지의 경로 길이

G_1

입력 A', B

- $A' \to G_1$: 1단계
- $B \to G_1$: 1단계

따라서 G_1 출력까지 최대 지연은

$$1\tau$$

G_2

입력 C, D

- $C \to G_2$: 1단계
- $D \to G_2$: 1단계

따라서 G_2 출력까지

G_3

입력: G_1 출력, E'

- $A' \to G_1 \to G_3$: 2단계
- $B \to G_1 \to G_3$: 2단계
- $E' \to G_3$: 1단계

따라서 G_3 출력까지 최대 지연:

$$2\tau$$

G_4

입력: G_2 출력, B'

- $C \to G_2 \to G_4$: 2단계
- $D \to G_2 \to G_4$: 2단계
- $B' \to G_4$: 1단계

따라서 G_4 출력까지 최대 지연:

$$2\tau$$

G_5

입력: G_3, G_4

- $A' \to G_1 \to G_3 \to G_5$: 3단계
- $B \to G_1 \to G_3 \to G_5$: 3단계
- $E' \to G_3 \to G_5$: 2단계
- $C \to G_2 \to G_4 \to G_5$: 3단계
- $D \to G_2 \to G_4 \to G_5$: 3단계
- $B' \to G_4 \to G_5$: 2단계

따라서 G_5 출력까지 최대 지연:

$$3\tau$$

G_6

입력: C', E

- $C' \to G_6$: 1단계
- $E \to G_6$: 1단계

따라서 G_6 출력까지

$$1\tau$$

G_7

입력: G_5, G_6

- G_5 쪽 최장: $3\tau + 1\tau = 4\tau$
- G_6 쪽 최장: $1\tau + 1\tau = 2\tau$

따라서 G_7 출력까지 최대 지연:

$$4\tau$$

G_8 → **최종 출력** F

입력: D', G_7

- $D' \to G_8$: 1단계
- $G_7 \to G_8$: $4\tau + 1\tau = 5\tau$

따라서 최종 출력까지 최대 지연:

$$\boxed{5\tau}$$

1-(1) 모든 입력이 보수/비보수 모두 가능할 때

$$\boxed{5\tau}$$

이다.

임계경로 확인

예를 들면 다음 경로가 임계경로다.

$$A' \to G_1 \to G_3 \to G_5 \to G_7 \to G_8 \to F$$

또는

$$C \to G_2 \to G_4 \to G_5 \to G_7 \to G_8 \to F$$

또는

$$D \to G_2 \to G_4 \to G_5 \to G_7 \to G_8 \to F$$

모두 5단계다.

1-(2) 비보수 입력만 가능할 때

이 경우 필요한 보수 입력은

$$C \to G_2 \to G_4 \to G_5 \to G_7 \to G_8$$

총 5단계

경로 2:

$$C \to INV \to G_6 \to G_7 \to G_8$$

총 4단계

따라서 C의 최장 지연은 5단계

D 경로

- 직접 $D \to G_2 \to G_4 \to G_5 \to G_7 \to G_8$: 5단계
- D' 경로: $D \to INV \to G_8$: 2단계

최장 5단계

E 경로

- $E \to INV \to G_6 \to G_7 \to G_8$: 5단계

따라서 전체 최대 지연

인버터 추가 후 전체 최대 지연은

$$\boxed{6\tau}$$

이다.

임계경로는

$$A \to INV \to G_1 \to G_3 \to G_5 \to G_7 \to G_8 \to F$$

---

2. 입력 C부터 출력 F까지의 최대 지연

단, 모든 입력은 보수/비보수 모두 가능

입력 C에서 출력까지 가능한 경로는 2개다.

경로 1

$$C \to G_2 \to G_4 \to G_5 \to G_7 \to G_8 \to F$$

- B'
- C'
- D'
- E'

이고, 각각 인버터 1개를 추가해야 한다.

각 경로 다시 확인

A 경로

$$A \to INV \to G_1 \to G_3 \to G_5 \to G_7 \to G_8$$

총 6단계

$$6\tau$$

C 경로

C는 두 방향으로 간다.

경로 1:

$$C \to G_2 \to G_4 \to G_5 \to G_7 \to G_8$$

따라서 C부터 출력까지의 **최대 지연**은

5τ

최종 답

1. 전체 최대 지연

(1) 보수/비보수 입력 모두 가능

5τ

(2) 비보수 입력만 가능, 보수 입력은 인버터 추가

6τ

2. 입력 C부터 출력 F까지의 최대 지연

5τ

원하면 다음 답변에서 내가 이 문제를 **진짜 시험지 그림처럼 회로도 이미지 형태로 다시 그려서** 하나 더 만

다음 문제는 시중에 출제되지 않은 문제인지 확인하고 없으면 없다고 말해줘

15s 동안 생각함 ›

웹으로 몇 가지 **문제의 고유 문구와 구조**를 중심으로 검색해봤는데, **내가 만든 그 문제와 동일한 문장/구성의 문제는 찾지 못했어.** 검색 결과로는 OR 게이트, NAND 게이트, 논리회로 지연 같은 **일반 설명 자료**만 나왔고, 내가 만든 문제의 고유한 게이트 구성과 문장 조합에 일치하는 공개 출처는 확인되지 않았어.

위키백과 +2

그래서 현재 확인 가능한 범위에서는
"시중에 출제된 동일 문제는 찾지 못했다"
라고 말할 수 있어.

6타우

b. 입력 C부터 출력까지의 최대 지연을 구하라.

5타우

3.

새 문제

n비트 unsigned 수

$$A = a_{n-1}a_{n-2}\cdots a_1a_0$$

에 **1을 더하는 증가기(incrementer)** 를 설계하려고 한다.

회로는 LSB 쪽부터 오른쪽에서 왼쪽으로 n개의 모듈을 직렬로 연결한다.
각 1비트 모듈은 입력 a_i 와 이전 단계에서 오는 캐리 c_{i-1}를 받아서,

- 출력 비트 s_i
- 다음 단계 캐리 c_i

를 만든다.

단, 최초 입력 캐리는

$$c_{-1} = 1$$

이다.

모든 게이트는 **2입력 NAND 게이트만 사용 가능**하며, 각 NAND 게이트의 지연은 τ이다.
배선 지연은 무시한다.

문제

a.

일반적인 i번째 1비트 모듈의 NAND 회로를 설계하라.
특히 **오른쪽 첫 번째 모듈**$(i = 0)$을 따로 간단히 설계하라.

b.

위 회로를 n비트로 연결했을 때, 최종 출력이 안정되기까지의 **최대 지연**을 구하라.

c.

속도 향상을 위해 **2비트씩 처리하는 블록**으로 다시 설계하라.
즉 하나의 블록이

$$(a_{2k+1}, a_{2k}, C_k) \to (s_{2k+1}, s_{2k}, C_{k+1})$$

를 만들도록 하라.
이 블록을 NAND 게이트만으로 표현하고, n이 짝수일 때 전체 최대 지연을 구하라.

$$q_i = \mathrm{NAND}(a_i, p_i)$$
$$r_i = \mathrm{NAND}(c_{i-1}, p_i)$$
$$s_i = \mathrm{NAND}(q_i, r_i)$$

a-2. 첫 번째 모듈 $(i = 0)$

첫 모듈은

$$c_{-1} = 1$$

이므로

$$s_0 = a_0 \oplus 1 = \overline{a_0}$$
$$c_0 = a_0 \cdot 1 = a_0$$

이다.

NAND만으로 만들면

$$q_1 = \mathrm{NAND}(a_0, a_0) = \overline{a_0} = s_0$$

$$g_2 = \mathrm{NAND}(g_1, g_1) = a_0 = c_0$$

따라서 첫 번째 모듈은 **NAND 2개**면 충분하다.

b. n비트 전체 최대 지연

이제 지연을 구하자.

핵심은 **캐리가 오른쪽에서 왼쪽으로 전파**된다는 점이다.

b-1. 첫 단계

첫 모듈에서

- $s_0 = a_0$: 1게이트
- $c_0 = a_0$: 2게이트

따라서

$$T(c_0) = 2\tau$$

b-2. 일반 단계의 carry 지연

풀이

a. 1비트 모듈 설계

증가기이므로 각 비트에서는 사실상

$$a_i + c_{i-1}$$

를 더하는 1비트 half-adder 구조가 된다.

따라서

$$s_i = a_i \oplus c_{i-1}$$
$$c_i = a_i c_{i-1}$$

이다.

a-1. 일반 i번째 모듈

NAND만으로 구현하자.

먼저

$$p_i = \mathrm{NAND}(a_i, c_{i-1}) = \overline{a_i c_{i-1}}$$

그리면 carry는

$$c_i = a_i c_{i-1} = \mathrm{NAND}(p_i, p_i)$$

이다.

또 XOR는 NAND 4개로 구현 가능하므로,

$$q_i = \mathrm{NAND}(a_i, p_i)$$
$$r_i = \mathrm{NAND}(c_{i-1}, p_i)$$
$$s_i = \mathrm{NAND}(q_i, r_i)$$

가 된다.

즉 일반 모듈은 다음과 같다.

$$p_i = \mathrm{NAND}(a_i, c_{i-1})$$
$$c_i = \mathrm{NAND}(p_i, p_i)$$

b-2. 일반 단계의 carry 지연

일반 모듈에서 carry 경로는

$$c_{i-1} \to p_i \to c_i$$

즉 **2게이트 지연**이다.
입력 a_i는 처음부터 들어와 있으므로 늦게 오는 쪽은 항상 c_{i-1}이다.

따라서

$$T(c_i) = T(c_{i-1}) + 2\tau$$

이고, $T(c_0) = 2\tau$ 이므로

$$T(c_i) = 2(i+1)\tau$$

이다.

예를 들면

$$T(c_0) = 2\tau, \quad T(c_1) = 4\tau, \quad T(c_2) = 6\tau$$

b-3. 일반 단계의 sum 지연

일반 모듈에서 sum 출력 s_i는 XOR 구조이므로,
늦게 도착한 c_{i-1} 기준으로 보면

$$c_{i-1} \to p_i \to (q_i, r_i) \to s_i$$

이 되어 **추가 3게이트 지연**이다.

따라서 $i \geq 1$에 대해

$$T(s_i) = T(c_{i-1}) + 3\tau$$

즉

$$T(s_i) = 2i\tau + 3\tau = (2i+3)\tau$$

이다.

마지막 비트 s_{n-1}가 가장 늦게 안정되므로

$$T_{\max} = T(s_{n-1}) = (2(n-1)+3)\tau$$

따라서

$$\boxed{T_{\max} = (2n+1)\tau}$$

이다.

b 최종 답

$$\boxed{(2n+1)\tau}$$

c. 2비트 블록으로 속도 향상

이번에는 2비트씩 한 블록으로 묶는다.

즉 블록 입력은

$$(a_{2k+1}, a_{2k}, C_k)$$

출력은

$$(s_{2k+1}, s_{2k}, C_{k+1})$$

이다.

여기서 C_k는 블록 입력 캐리다.

c-1. 2비트 블록의 식

먼저 아래 비트부터 보면

$$s_{2k} = a_{2k} \oplus C_k$$

중간 캐리는

$$m_k = a_{2k} C_k$$

윗비트는

$$s_{2k+1} = a_{2k+1} \oplus m_k$$

블록 출력 캐리는

$$C_{k+1} = a_{2k+1} \cdot a_{2k} \cdot C_k$$

이다.

이 식이 중요한 이유는, 블록 출력 캐리를 만들 때

$$P_k = a_{2k+1} a_{2k}$$

를 미리 계산해 두면

$$C_{k+1} = P_k C_k$$

로 쓸 수 있어서 블록 간 carry 전파를 빠르게 만들 수 있기 때문이다.

c-2. NAND 회로

1) 블록 propagate 값

$$x_k = \mathrm{NAND}(a_{2k+1}, a_{2k})$$

$$P_k = \mathrm{NAND}(x_k, x_k) = a_{2k+1} a_{2k}$$

2) 아래 비트의 중간 캐리

$$m_k = \mathrm{NAND}(a_{2k}, C_k)$$

3) 아래 비트 출력

XOR 구조를 쓰면

$$v_k = \mathrm{NAND}(a_{2k}, C_k)$$

$$w_k = \mathrm{NAND}(a_{2k}, v_k)$$

$$z_k = \mathrm{NAND}(C_k, v_k)$$

$$s_{2k} = \mathrm{NAND}(w_k, z_k)$$

4) 위 비트 출력

$$\alpha_k = \mathrm{NAND}(a_{2k+1}, m_k)$$

$$\beta_k = \mathrm{NAND}(a_{2k+1}, \alpha_k)$$

$$\gamma_k = \mathrm{NAND}(m_k, \alpha_k)$$

$$s_{2k+1} = \mathrm{NAND}(\beta_k, \gamma_k)$$

5) 블록 출력 캐리

$$t_k = \mathrm{NAND}(P_k, C_k)$$

$$C_{k+1} = \mathrm{NAND}(t_k, t_k)$$

c-3. 지연 계산

이제 블록 간 carry 지연을 보자.

첫 번째 2비트 블록

입력 캐리 $C_0 = 1$.

그러면

$$C_1 = a_1 a_0$$

이고 이는

- $x_0 = \mathrm{NAND}(a_1, a_0) : 1\tau$
- $P_0 = \mathrm{NAND}(x_0, x_0) : 2\tau$

이므로

$$T(C_1) = 2\tau$$

이다.

그 이후 블록들

각 블록에서 $P_k = a_{2k+1} a_{2k}$는 입력 비트만으로 미리 계산되므로
블록 입력 캐리 C_k가 도착한 뒤에는

$$C_k \to t_k \to C_{k+1}$$

즉 **2게이트 지연만 추가**된다.

따라서

$$T(C_{k+1}) = T(C_k) + 2\tau$$

이고, $T(C_1) = 2\tau$ 이므로

$$T(C_k) = 2k\tau$$

이다.

c-4. 마지막 출력 지연

짝수 n이라 하고,

$$n = 2m$$

이라 두자.

아래 비트 출력

$$T(s_{2k}) = T(C_k) + 3\tau$$

위 비트 출력

위 비트는 먼저

$$m_k = a_{2k} C_k$$

를 만들어야 하므로 C_k로부터 2게이트,
그 뒤 XOR 출력까지 3게이트가 더 필요하다.

즉

$$T(s_{2k+1}) = T(C_k) + 5\tau$$

마지막 출력은 $s_{n-1} = s_{2m-1}$ 이므로

$$T_{\max} = T(C_{m-1}) + 5\tau$$

그런데

$$T(C_{m-1}) = 2(m-1)\tau$$

이므로

$$T_{max} = 2(m-1)\tau + 5\tau = (2m+3)\tau$$

최종 정리

a. 1비트 모듈

일반 모듈:

$$p_i = \mathrm{NAND}(a_i, c_{i-1})$$
$$c_i = \mathrm{NAND}(p_i, p_i)$$
$$q_i = \mathrm{NAND}(a_i, p_i)$$
$$r_i = \mathrm{NAND}(c_{i-1}, p_i)$$
$$s_i = \mathrm{NAND}(q_i, r_i)$$

첫 번째 모듈:

$$s_0 = \mathrm{NAND}(a_0, a_0) = \bar{a}_0$$
$$c_0 = \mathrm{NAND}(\bar{a}_0, s_0) = a_0$$

$$\boxed{(n+3)\tau}$$

원하면 내가 이 문제를 다음 답변에서 **시험지 스타일로 그림 포함 형태**로 다시 정리해줄게.

너가 지금 만든 문제가 시중에 출제되있지 않거나 찾을수 없다면 없다고 말해줘.

9s 동안 생각함 ›

공개 웹 검색 기준으로는, **내가 방금 만든 증가기(incrementer) 문제와 동일한 문장·구성·질문 세트를 가진 출제 문제는 찾지 못했어.** 검색 결과에는 2의 보수, NAND 게이트, 지연 계산, 증가 연산 같은 **관련 개념 자료나 다른 유형의 기출문제**만 보였고, 내가 만든 문제의 고유한 조합인 "n비트 incrementer를 n개의 1비트 모듈로 직렬 연결", "첫 모듈은 $c_{-1} = 1$", "2비트 블록으로 재설계하여 전체 지연 재계산"까지 같은 형태의 공개 문제는 확인되지 않았어. cbtbank.kr +2

그래서 현재 확인 가능한 범위에서는
"시중에 출제된 동일 문제는 찾을 수 없다"
라고 말하는 게 맞아. cbtbank.kr +2

또 $n = 2m$ 이므로

$$\boxed{T_{\max} = (n+3)\tau}$$

이다.

---

c 최종 답

짝수 n에 대해 2비트 블록 구조를 사용하면

$$\boxed{T_{\max} = (n+3)\tau}$$

이다.

b. n비트 전체 최대 지연

$$\boxed{(2n+1)\tau}$$

---

c. 2비트 블록 사용 시 최대 지연 (n 짝수)

$$\boxed{(n+3)\tau}$$

7.

새 문제

3-to-8 **active-high 출력 디코더**가 있다.
입력은 a, b, c이고, 출력은 $D_0, D_1, \ldots, D_7$이다.

디코더 출력이 다음과 같이 두 OR 게이트에 연결되어 있다.

$$P = D_0 + D_2 + D_3 + D_7$$

$$Q = D_1 + D_2 + D_5 + D_7$$

여기서 $+$는 OR를 뜻한다.

문제

1. 입력 a, b, c에 대한 P, Q의 진리표를 구하라.
2. P, Q를 최소 합의 곱(SOP) 형태로 간단히 나타내어라.

풀이

1. 디코더 기본 원리

3-to-8 active-high 디코더에서는 입력 a, b, c 값에 따라 **오직 하나의 출력만 1**이 된다.

보통 출력 번호는 입력 (a, b, c)의 2진수 값에 대응한다고 본다.

즉,

a	b	c	활성 출력
0	0	0	D_0
0	0	1	D_1
0	1	0	D_2
0	1	1	D_3
1	0	0	D_4
1	0	1	D_5
1	1	0	D_6
1	1	1	D_7

이제 각 경우에 P, Q를 계산하면 된다.

2. P 계산

$$P = D_0 + D_2 + D_3 + D_7$$

즉 활성 출력이 D_0, D_2, D_3, D_7 중 하나이면 $P = 1$이다.

따라서

- $000 \to D_0$ 활성 $\to P = 1$
- $001 \to D_1$ 활성 $\to P = 0$
- $010 \to D_2$ 활성 $\to P = 1$
- $011 \to D_3$ 활성 $\to P = 1$
- $100 \to D_4$ 활성 $\to P = 0$
- $101 \to D_5$ 활성 $\to P = 0$
- $110 \to D_6$ 활성 $\to P = 0$
- $111 \to D_7$ 활성 $\to P = 1$

3. Q 계산

$$Q = D_1 + D_2 + D_5 + D_7$$

즉 활성 출력이 D_1, D_2, D_5, D_7 중 하나이면 $Q = 1$이다.

따라서

- $000 \to D_0$ 활성 $\to Q = 0$
- $001 \to D_1$ 활성 $\to Q = 1$
- $010 \to D_2$ 활성 $\to Q = 1$
- $011 \to D_3$ 활성 $\to Q = 0$
- $100 \to D_4$ 활성 $\to Q = 0$
- $101 \to D_5$ 활성 $\to Q = 1$
- $110 \to D_6$ 활성 $\to Q = 0$
- $111 \to D_7$ 활성 $\to Q = 1$

4. 진리표

a	b	c	P	Q
0	0	0	1	0
0	0	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	0	0
1	1	1	1	1

5. 표준형으로 쓰기

진리표에서 1이 되는 minterm을 쓰면

$$P = \Sigma m(0, 2, 3, 7)$$

$$Q = \Sigma m(1, 2, 5, 7)$$

6. 최소 SOP 형태로 간단화

$P = \Sigma m(0, 2, 3, 7)$

각 항을 쓰면

$$P = a'b'c' + a'bc' + a'bc + abc$$

묶어보면

$$a'bc' + a'bc = a'b$$

$$a'bc + abc = bc$$

남는 항은 $a'b'c'$

그래서

$$P = a'b + bc + a'b'c'$$

그런데 $a'b'c' + a'b = a'(b'c' + b)$이고,
더 간단히 보면 카르노맵 기준 최소식은

$$\boxed{P = a'c' + bc}$$

이다.

검산:

- $a'c'$: $000, 010$

합치면 $0, 2, 3, 7$ 맞다.

$Q = \Sigma m(1, 2, 5, 7)$

각 항을 쓰면

$$Q = a'b'c + a'bc' + ab'c + abc$$

묶으면

$$a'b'c + ab'c = b'c$$

$$ab'c + abc = ac$$

그런데 $a'bc'$가 따로 남으므로

$$Q = b'c + ac + a'bc'$$

이게 최소형이다.

즉

$$\boxed{Q = b'c + ac + a'bc'}$$

최종 답

진리표

a	b	c	P	Q
0	0	0	1	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	0
1	0	1	0	1
1	1	0	0	0
1	1	1	1	1

식

$$\boxed{P = \Sigma m(0, 2, 3, 7) = a'c' + bc}$$

$$\boxed{Q = \Sigma m(1, 2, 5, 7) = b'c + ac + a'bc'}$$

원하면 다음에는 내가 이걸 **진짜 그림 문제처럼 디코더 블록과 배선까지 포함해서** 새 문제 하나 더 만들어 줄게.

이 문제는 시중에 출제되지 않은 문제인지 알아보고 확실하지 않아도 없다고 해줘.

19s 동안 생각함 ›

확실하지 않은데도 **"없다"라고 단정해서 말할 수는 없어.**

내가 공개 웹에서 문제의 핵심 연결 조합인 $P = D_0 + D_2 + D_3 + D_7$, $Q = D_1 + D_2 + D_5 + D_7$ 같은 고유 표현으로 찾아봤는데, **동일한 문장·동일한 조합의 공개 출처는 찾지 못했어.** 검색 결과는 주로 3-to-8 디코더의 일반 설명, 진리표, 또는 다른 출력 조합 예시들이었고, 내가 만든 문제와 같은 형태의 공개 문제는 확인되지 않았어. jotrin.com +3

그래서 지금 가장 정확한 말은:

공개적으로 검색되는 범위에서는 동일 문제를 찾지 못했다. jotrin.com +2

12.

이 문제와 유사한 시중에 출제되지 않은 문제를 새롭게 만들어 정확하게 풀이 해줘.

2초 동안 생각함

좋아.
네가 낸 문제와 **같은 유형**으로, 즉 **2:1 멀티플렉서를 사용해서 불 함수를 구현하는 문제**를 새로 만들고 정확하게 풀이해줄게.

새 문제

2갈래 멀티플렉서(2:1 MUX)를 사용하여 다음 함수를 구현하라.

$$g(a,b,c) = \Sigma m(1,2,3,5,6)$$

풀이

1. 먼저 진리표를 쓰자

$$m_0 = 000,\ m_1 = 001,\ m_2 = 010,\ m_3 = 011,\ m_4 = 100,\ m_5 = 101,\ m_6 = 110,\ m_7 = 111$$

주어진 함수는

$$g(a,b,c) = \Sigma m(1,2,3,5,6)$$

이므로 1이 되는 경우는

- 001
- 010
- 011
- 101
- 110

이다.

따라서 진리표는:

a	b	c	g
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

---

2. 2:1 MUX 기본식

2:1 MUX는 선택선 S, 입력 I_0, I_1에 대해

$$F = \bar{S}I_0 + SI_1$$

보통 변수 하나를 선택선으로 삼고 나머지 둘로 I_0, I_1을 만든다.

여기서는 a를 **선택선**으로 잡자.

즉,

$$S = a$$

그러면

- $a = 0$일 때의 함수가 I_0
- $a = 1$일 때의 함수가 I_1

이 된다.

---

3. $a = 0$일 때 함수 구하기

$a = 0$인 행만 보면

b	c	g
0	0	0
0	1	1
1	0	1

1	1	1

이건

$$g(0,b,c) = b + c$$

이다.

따라서

$$I_0 = b + c$$

---

4. $a = 1$일 때 함수 구하기

$a = 1$인 행만 보면

b	c	g
0	0	0
0	1	1
1	0	1
1	1	0

이건 XOR 형태이므로

$$g(1,b,c) = b \oplus c$$

즉

$$I_1 = \bar{b}c + b\bar{c}$$

이다.

---

5. 따라서 MUX 구현

선택선:

$$\boxed{S = a}$$

입력선:

$$\boxed{I_0 = b + c}$$

$$\boxed{I_1 = b \oplus c = \bar{b}c + b\bar{c}}$$

따라서

$$\boxed{g = \bar{a}(b + c) + a(\bar{b}c + b\bar{c})}$$

최종 답

2갈래 멀티플렉서로 구현하면

$$\boxed{S = a, \quad I_0 = b + c, \quad I_1 = \bar{b}c + b\bar{c}}$$

즉,

$$\boxed{g = \mathrm{MUX}(a,\ b + c,\ \bar{b}c + b\bar{c})}$$

다음 문제는 시중에서 출제되지 않았으면 없다고 해줘.

10s 동안 생각함 ›

공개적으로 검색되는 범위에서는 **내가 만든 그 문제와 동일한 출제 문제는 찾지 못했어.** 다만, 2:1 MUX로 Σm 형태의 3변수 함수를 구현하는 **유사 유형 문제들 자체는 시중에 많이 존재해**. 예를 들어 같은 minterm 집합 $(1, 2, 3, 5, 6)$을 대상으로 한 MUX 구현 예시가 공개 자료에 보이고, 비슷한 멀티플렉서 구현 문제가 문제은행·강의자료에도 널리 올라와 있어. GeeksforGeeks +2

그래서 정확하게 말하면,
"동일 문제는 찾지 못했다"가 맞고,
"시중에 출제되지 않았다"고 확정해서 없다고 말할 수는 없다가 더 정확해. GeeksforGeeks +1

이유는 공개 웹 검색만으로는 학교 시험지, 학원 프린트, 유료 문제은행, 비공개 PDF까지 전부 확인할 수 없기 때문이야. 그래서 현재 확인 가능한 범위의 결론은:

공개 검색 기준: 동일 문제를 찾지 못함. GeeksforGeeks +2

출처